



FOREWORD

A Microsoft Technical Fellow once told me he had never met someone who understood how the security of the Windows operating system actually worked. While I don't think he was right (and plan to send him a copy of this book to prove it), he had a point. Though critical, there is no doubt that Windows security is complex.

One of the reasons for this is related to the core architectural difference between Linux and Windows. Linux is a file-oriented operating system, while Windows is API oriented, and though APIs can provide a much richer set of capabilities, they come at the expense of simplicity. So, exploring an API-oriented operating system is more difficult. You need to read the API documentation, write code, compile and run it, and debug the results.

This is a very time-consuming loop, and it's why so few people have a deep understanding of how Windows security works—it's just too hard to explore.

It was because of these problems that I invented PowerShell. I wanted administrators to automate Windows and had originally tried to do so by distributing Unix tools for free. (Remember Windows Services for Unix?) This failed because Unix tools work on files, while everything important in Windows lives behind an API. Thus, `awk` didn't work against the registry, `grep` didn't work against Windows Management Instrumentation (WMI), `sed` didn't work against Active Directory, and so on. What we needed was an API-oriented command line interface and scripting tool. So, I created PowerShell.

Today, James is using PowerShell to address the difficulty of acquiring Windows security expertise; he has made the system explorable. Step one: install his PowerShell module, `NTObjectManager`, which provides over 550 cmdlets to experiment with all aspects of Windows security. This hands-on exploration will allow you to understand how things really work.

This book belongs on the desk of every security professional and developer working with Windows security. **Part I** provides an overview of Windows security's architecture, **Part II** covers the details of the operating system's security mechanisms and services, and **Part III** explores the various aspects of Windows authentication. Each chapter includes a set of PowerShell examples.

I strongly encourage you to follow the examples provided; exploration turns words into experience, and experience is the foundation of competence. Run the commands, make intentional mistakes, and see what errors you get. In doing so, you'll acquire a deep understanding of the system.

And trust me: it will be fun.

Jeffrey Snover

Inventor of PowerShell, former chief architect for Windows Server,
and former Microsoft Technical Fellow